40 Days of LeetCode Challenge

40 days, 40 problems
DAY :12

LEETCODE PROBLEM NO. 217 : Contains Duplicate

Problem Link : https://leetcode.com/problems/contains-duplicate/
Arrays · HashSet · Boolean return value · array traversal · time-space complexity.

STEP BY STEP SOLUTION

Step 1. Understand the question :-

We are given an integer array nums .Our task is to determine whether any element appears more than once.

- If even one element appears at least twice, return true.
- If every element appears only once, return false.

NOTE:-

We don't need to find the duplicate or its index. We only need to check whether a duplicate exists.

Step 2. Translating it into simpler language with examples :-

Example 1:
Input: nums = [1,2,3,1]
Output: true
Explanation:
The element 1 occurs at the indices 0 and 3.

Example 2:
Input: nums = [1,2,3,4]
Output: false
Explanation:
All elements are distinct.

Example 3:
Input: nums = [1,1,1,3,3,4,3,2,4,2]
Output: true

Step 3. Identify the pattern

Hashing — HashSet

A HashSet is a Java collection that stores only unique elements. For example:

HashSet<Integer> set = new HashSet<>();

set.add(1);
set.add(2);
set.add(3);
set.add(1);

The set will contain: [1, 2, 3]

NOTE:-

Notice that the second occurrence of 1 is not added. This gives us a useful property:

- If an element is successfully added, it wasn't already present.
- If an element cannot be added, it already exists in the set.

Step 4. Build the logic

Here's the approach we'll follow.

Algorithm

1. Create an empty HashSet named uniqueNumbers.
2. Traverse the given array using an enhanced for loop.
3. For every element, attempt to add it to the HashSet.
4. If add() returns false, the element already exists in the set. Return true.
5. If the loop completes without finding a duplicate, return false.

Pseudocode

```
START

  Create an empty HashSet called uniqueNumbers

  FOR each number in nums:

    IF number cannot be added to uniqueNumbers:

      RETURN true

    END IF

  END FOR

  RETURN false

END
```

Step 5. Code in Java

```
import java.util.*;

class Solution {

  public boolean containsDuplicate(int[] nums) {

    Set<Integer> uniqueNumbers = new HashSet<>();

    for (int num : nums) {

      if (!uniqueNumbers.add(num)) {

        return true; }
```

```
        }
 return false;
    }
}
```

How to submit

1. Open LeetCode Problem [217 : Contains Duplicate]
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

A. *Import Java utilities*

```
import java.util.*;
```

Imports Java utility classes, including Set and HashSet.

B. *Class declaration*

```
class Solution {
```

Defines the class required by LeetCode.

C. *Method declaration*

```
public boolean containsDuplicate(int[] nums) {
```

- public: The method can be accessed by LeetCode's testing system.
- boolean: The method returns either true or false.
- containsDuplicate: Method name.
- int[] nums: Input integer array.

D. *Creating the HashSet*

```
Set<Integer> uniqueNumbers = new HashSet<>();
```

Part	Meaning
Set<Integer>	A collection that stores unique integers
uniqueNumbers	Name of our set
new HashSet<>()	Creates an empty HashSet

NOTE:-

Why use Set instead of HashSet on the left side?

Because Set is the interface and HashSet is its implementation. This is a common Java programming . practice.

E. *Enhanced for loop*

for (int num : nums) {

Traverses every element of the array one by one. For example, if: nums = [1, 2, 3, 1]

Then num takes the values: 1 → 2 → 3 → 1

F. *Checking for duplicates*

if (!uniqueNumbers.add(num)) {

The add() method of a HashSet returns:

- true if the element is successfully added.
- False if the element already exists.
-

The ! operator reverses the boolean result.

Therefore:

!uniqueNumbers.add(num)

becomes true when the element is already present.

For example:

```
uniqueNumbers.add(1);  // true
uniqueNumbers.add(2);  // true
uniqueNumbers.add(1);  // false
```

When the second 1 is encountered, the condition becomes true.

G. *Return true*

return true;

As soon as we detect a duplicate, we return true .There is no need to check the remaining elements because th e answer is already known.

H. *Closing the loop*

}

If no duplicate is found, the loop continues until all elements are checked.

I. *Return false*

return false;

If the loop finishes without returning true, every element is unique. Therefore, we return false.

Dry run

Let's take Example 1:

nums = [1, 2, 3, 1]

Initially: uniqueNumbers = {}

Iteration	num	add(num) result	Set after operation	Action
1	1	true	{1}	Continue
2	2	true	{1, 2}	Continue
3	3	true	{1, 2, 3}	Continue
4	1	false	{1, 2, 3}	Return true

Final output = Duplicate found :- true

The second occurrence of 1 is detected, so the method terminates immediately.

Step 7. Time complexity & Auxiliary space complexity

Complexity	Value	Explanation
Time	O(n) average	Each element is processed once, and HashSet operations take O(1) average time.
Space	O(n)	In the worst case, all n elements are unique and stored in the set.

Best case: O(1) — a duplicate is found immediately.

Worst case: O(n) — all elements are unique, so we process the entire array.

NOTE:-

three alternatives and compare them with your HashSet solution.

Approach 1 – Brute force (nested loops)

Compare every element with all the elements that come after it.

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] == nums[j]) {
                    return true;
                }
            }
        }

        return false;
    }
}
```

Logic:

- The outer loop selects one element.
- The inner loop compares it with the remaining elements.
- If a match is found, return true.
- If no pair matches, return false.

Example:

nums = [1, 2, 3, 1]

Compare 1 with 2 → Different

Compare 1 with 3 → Different

Compare 1 with 1 → Same → true

Complexity:

- Time: O(n^2)
- Space: O(1)
- This approach is easy to understand but inefficient for large arrays.

Approach 2 – Sorting

Sort the array first. If duplicates exist, they will become adjacent.

```
import java.util.Arrays;

class Solution {
    public boolean containsDuplicate(int[] nums) {
        Arrays.sort(nums);

        for (int i = 1; i < nums.length; i++) {
            if (nums[i] == nums[i - 1]) {
                return true;
            }
        }

        return false;
    }
}
```

Logic:

Suppose:

Original: [4, 2, 1, 3, 2]

After sorting: [1, 2, 2, 3, 4]

Now compare neighboring elements:

1 and 2 → Different

2 and 2 → Same → true

Complexity:

- Time: O(n log n) for sorting, followed by O(n) scanning.
- Auxiliary space: Depends on the sorting implementation;
- Java's primitive-array sort uses an implementation-specific workspace.
- This approach modifies the original array because Arrays.sort() sorts it in place.

Approach 3 – HashSet (your approach)

```
import java.util.*;

class Solution {
   public boolean containsDuplicate(int[] nums) {
      Set<Integer> uniqueNumbers = new HashSet<>();

      for (int num : nums) {
         if (!uniqueNumbers.add(num)) {
            return true;
         }
      }

      return false;
   }
}
```

Logic: Store each number in a set. If an insertion fails, a duplicate has been found.

Complexity:

- Time: O(n) average.
- Space: O(n)

Comparison of all approaches

Approach	Time complexity	Space complexity	Modifies input?
Brute force	$O(n^2)$	O(1)	No
Sorting	O(n log n)	Depends on sorting implementation	Yes
HashSet	O(n) average	O(n)	No

SCREENSHOT:-

